

eFuse / OTP Behavioral Model Specification

Version: 2.0

Status: Specification

Audience: DV users, SoC integrators, and simulation maintainers

License: Apache-2.0

1. Purpose

This document specifies the behavior of a reusable eFuse / OTP model for digital simulation. It defines the logical storage format, command protocol, programming semantics, failure-injection behavior, and reset/readback behavior.

The model is functional. It represents the digital contract needed by a controller or testbench and does not attempt to model physical OTP effects.

2. Scope

2.1 Supported Behavior

Capability	Support
OTP storage array	8192 bits, modeled as 256 x 32-bit words
Image preload	\$readmemh-style 32-bit word image
Fuse READ	Multi-beat 32-bit word stream
Fuse PROGRAM	Sticky OR of one programmable bit
PROGRAM_READ_BACK	PROGRAM plus updated-word response
Program failure injection	Deterministic count or seeded percent mode
Reset / reload	Reloads image and overlays successfully programmed bits
Optional bank-control access	Handshake-correct AXI-Lite responder; reads return zero

2.2 Non-Goals

Item	Status
Analog behavior	Not modeled
Silicon timing / pulse-width behavior	Not modeled
Sense margins, disturb, endurance, aging	Not modeled

Physical OTP macro details	Not modeled
Rich OTP-bank register behavior	Not modeled; bank reads return zero
Access-policy features such as read-lock/write-lock enforcement	Out of model scope unless added by an integrator

3. Port List

The model has no parameters. It exposes one clock/reset pair, a minimal bank-control AXI-Lite interface, and a fuse-command interface.

Port	Direction	Type	Description
clk_i	input	logic	Clock for all registered protocol behavior.
rst_ni	input	logic	Active-low reset. Resets protocol state and re-arms failure injection.
bank_ctrl_req_i	input	AXI-Lite request struct	Optional bank-control request channel.
bank_ctrl_resp_o	output	AXI-Lite response struct	Minimal registered response channel. Writes ACK OKAY; reads return zero.
fuse_command_req_i	input	Fuse command request struct	READ / PROGRAM / PROGRAM_READ_BACK command channel.
fuse_command_resp_o	output	Fuse command response struct	READ data stream or single-beat PROGRAM response.

3.1 Fuse Command Request

Field	Width	Description
address	13 bits	Fuse bit address. Word index is address >> 5; bit index is address[4:0].
program_data	1 bit	Bit value used by PROGRAM commands.
access_length_words	9 bits	Number of 32-bit READ beats to return.
command	2 bits	Command opcode.
valid	1 bit	Request valid. Must deassert between independent commands.

3.2 Fuse Command Response

Field

data

status

valid

3.3 Command Opcodes

Opcode

2'b00

2'b01

2'b10

other

4. Storage and Image Format

The OTP array is modeled as:

`otp_mem[0..255] : 32-bit words`

The image file is a 256-line hexadecimal file:

```
00000000
00000000
000000f0
...
```

Rules:

- One 32-bit word per line.
- Hexadecimal text, no 0x prefix required.
- Word `i` maps to `otp_mem[i]`.
- Within each word, bit 0 is the least-significant bit.
- Missing image file is allowed; unspecified contents remain zero.

The responder also maintains:

`programmed_mem[0..255]`

`programmed_mem` records successful frontdoor PROGRAM operations. On reset or resense, the model reloads the image file into `otp_mem`, then overlays `programmed_mem` with OR semantics:

`otp_mem[i] = image_word[i] | programmed_mem[i]`

This models OTP persistence across reset while still allowing tests to regenerate the base image between sense cycles.

5. Protocol Behavior

5.1 READ

READ returns a 32-bit word stream starting at the requested bit address. The request payload must stay stable until the command completes.

Timing chart:

Cycle	Request	Model action	Response
T0	valid=1, command=READ, address=A, length=N	Latch request.	valid=0
T1	Request payload held stable.	Read word A >> 5.	valid=1, data=W0
T2	Request payload held stable.	Read next word.	valid=1, data=W1
...	Request payload held stable.	Continue one word per cycle.	valid=1, data=...
Tn	Request payload held stable.	Read final word.	valid=1, data=W(N-1)
Tn+1	valid=0	Command complete.	valid=0

Notes:

- `access_length_words = 0` returns zero response beats.
- The model advances its internal address by 32 bits per beat.
- The response channel has no backpressure; every `resp.valid` beat is consumed.

5.2 PROGRAM / PROGRAM_READ_BACK Success

PROGRAM and PROGRAM_READ_BACK are single-beat operations. On success, the addressed bit is OR'd into both `otp_mem` and `programmed_mem`.

Timing chart:

Cycle	Request	Model action	Response
T0	valid=1, command=PROGRA M_READ_BACK, address=bit_ad dr, program_data=1	Decode word and bit index.	valid=0
T1	Request may deassert after	<code>otp_mem[word] = (1 << bit)</code> and	valid=1, status=0,

	issue.	programmed_mem[word] = (1 << bit).	data=updated_word
T2	valid=0	Command complete.	valid=0

PROGRAM and PROGRAM_READ_BACK have the same storage behavior. The practical difference is how an integrator interprets the returned data field.

5.3 PROGRAM / PROGRAM_READ_BACK Failure

When failure injection selects a PROGRAM command to fail, the model does not update storage. It returns an error response and the current word value.

Timing chart:

Cycle	Request	Model action	Response
T0	valid=1, command=PROGRAM_READ_BACK, address=bit_address, program_data=1	Failure selected by injection policy.	valid=0
T1	Request may deassert after issue.	Storage unchanged.	valid=1, status=1, data=current_word
T2	valid=0	Command complete.	valid=0

5.4 Retry Flow

A typical software or controller retry flow is:

Step	Operation	Expected result
1	PROGRAM bit K.	Failure may be injected; storage remains unchanged.
2	Read or poll completion status.	Done is observed with error.
3	Clear completion/error status.	Controller is ready for another command.
4	Retry PROGRAM bit K.	On success, storage is updated with sticky-OR semantics.
5	Read or poll completion status.	Done is observed without error.
6	READ or resense.	Updated OTP contents are returned.

6. Program-Failure Injection

Failure injection is optional and disabled by default. An implementation may provide configuration fields for:

Configuration

Fail count

Fail percent

Fail seed

Selection priority:

1. If count mode has remaining budget, fail and decrement the count.
2. Else if percent mode is non-zero, advance the LFSR and compare against the percentage threshold.
3. Else, succeed.

The failure budget and LFSR state are re-armed from configuration on reset. This makes repeated sense/resense cycles deterministic.

7. Request Stability Contract

The fuse-command response interface has no `ready`. The model therefore serves one command per `valid` assertion.

Rules:

- Deassert `valid` between independent commands.
- Keep `command`, `address`, `program_data`, and `access_length_words` stable while `valid` remains asserted.
- A held `READ` sweep is legal because the request payload stays unchanged.
- A changed payload while `valid` remains asserted is a contract violation and shall be reported as a model error.

8. Bank-Control AXI-Lite Stub

The bank-control interface is a registered minimal AXI-Lite slave:

- `AW` and `W` can arrive independently and in either order.
- `b_valid` is held until `b_ready`.
- `r_valid` is held until `r_ready`.
- Write response is `OKAY`.
- Read response is `OKAY` with zero data.

This interface exists to keep bank-control accesses from hanging. It is not a full OTP-bank register model.

9. Integration Guide

An integration should connect the model as a replacement for the OTP macro interface used by the digital design under test.

Step	Requirement
Clock/reset	Drive <code>clk_i</code> and active-low <code>rst_ni</code> from the simulation testbench.
OTP image	Provide a 256-word hex image when deterministic nonzero fuse contents are required.
Fuse command path	Connect the controller's fuse-command request and response signals directly to the model.
Bank-control path	Connect bank-control AXI-Lite signals if the controller can access that aperture.
Programming	Use <code>PROGRAM</code> or <code>PROGRAM_READ_BACK</code> commands for sticky-OR bit updates.
Failure injection	Enable fail-count or fail-percent configuration only in tests that need retry/error behavior.
Reset/resense	Expect the model to reload the image and overlay successful programmed bits after reset.

Integration rules:

- The request payload must remain stable while `valid` is asserted.
- The requester must deassert `valid` between independent commands.
- The response channel has no `ready`; each `resp.valid` beat is accepted.
- Unsupported commands shall produce an error response and shall not update OTP storage.
- Integrators that need access-policy behavior, detailed bank registers, or physical OTP timing must add those features outside this base model.

10. Verification Expectations

An integration should prove at least:

Check	Expected evidence
READ stream	Correct word count and ordered data from the image.
PROGRAM success	Storage updates by sticky OR and response returns updated word.
PROGRAM failure	Storage does not update and response returns <code>status=1</code> .
Retry	A failed <code>PROGRAM</code> can be retried and later succeeds.
Reset/resense	Reloaded image is overlaid with successful programmed bits.
Bank-control	AXI-Lite <code>valid/ready</code> stability for minimal read/write responses.

11. Revision History

Version	Date	Notes
2.0	2026-06-25	Generalized specification wording, concise protocol timing tables, and generic integration guidance.